

Simplified RC4 stream cipher v2 (decryption)

Project

Design and implement a simplified version of the RC4 stream cipher for the decryption of ciphertext. The (original) algorithm of RC4 stream cipher for encryption can be found at the following link:

<https://en.wikipedia.org/wiki/RC4>

As indicated, it essentially consists of two sub-routines: one for the setup of the encryption material (the Key-Scheduling Algorithm, KSA) and one that performs the encryption process (the Pseudo-Random Generation Algorithm, PRGA). The original version of RC4 algorithm foresees to accept as input a symmetric key whose length in bytes (indicated as *keylength*) can vary from 1 up to 256: for the simplified version of RC4 algorithm it is required to fix the key length in bytes to 32 (i.e. 32 bytes, or 256 bits), for which the operation of KSA algorithm

$$j = (j + S[i] + \text{key}[i \bmod \text{keylength}]) \bmod 256$$

becomes

$$j = (j + S[i] + \text{key}[i \bmod 32]) \bmod 256$$

In case of decryption, the KSA algorithm is the same, while the decryption process can be implemented by swapping the plaintext bytes with the ciphertext bytes, therefore, the module required to be implemented shall consist of two sub-modules that implement each one of the two following algorithms, respectively, reported as pseudo-C code:

- KSA algorithm:

```
for i = 0 to 255 {  
    S[i] = i  
}  
j = 0  
for i = 0 to 255 {  
    j = (j + S[i] + key[i mod 32]) mod 256  
    swap(S[i], S[j])  
}
```
- PRGA algorithm (for decryption):

```
i = 0  
j = 0  
while(1){ {  
    i = (i + 1) mod 256  
    j = (j + S[i]) mod 256  
    swap(S[i], S[j])  
    K[l] = S[(S[i] + S[j]) mod 256]  
    P[l] = C[l] ⊕ K[l]  
}
```

being,

$C[l]$ the l^{th} (input) byte of ciphertext.

$P[l]$ the l^{th} (output) byte of plaintext.

$\oplus$ is the XOR operator.

and to be applied to each byte of ciphertext, to generate the corresponding plaintext, one byte per time.

Additional design specifications

- The module shall have an asynchronous active-low reset port.
- The module interface shall include an input flag to be driven as it follows: 1'b1, when input plaintext byte on the corresponding input port is valid and stable (i.e. it can be used by the internal module logic), 1'b0, otherwise.
- The module interface shall include an output flag to be driven as it follows: 1'b1, when output ciphertext byte on the corresponding output port is ready and stable (i.e., external modules can read and use it), 1'b0, otherwise.